SHERLOCK

# Security Review For Lombard Finance

Collaborative Audit Prepared For: **Lombard Finance**
Lead Security Expert(s): **deadmanwalking**
**xiaoming90**
Date Audited: **December 15 - December 18, 2025**

# Introduction

This update review focused on small updates on Lombard Finance's ERC4626 Vault Wrapper contract.

## Scope

Repository: lombard-finance/smart-contracts

Audited Commit: 19b96a287fc01ca795cabecf5093672ba34d5299

Final Commit: b23211fbfea441d500b57f1e490581a8d2dd4e81

Files:

- contracts/stakeAndBake/depositor/veda/ERC4626VaultWrapper.sol

## Final Commit Hash

**b23211fbfea441d500b57f1e490581a8d2dd4e81**

## Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

## High Severity Issues: 0

## Medium Severity Issues: 3

## Low/Info Severity Issues: 4

## Issues Not Fixed and Not Acknowledged

| High | Medium | Low/Info |
| --- | --- | --- |
| 0 | 0 | 0 |

# Issue M-1: 1:1 redemption invariant can be broken [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/3

## Vulnerability Detail

Redemption of the wrapper share is on a 1:1 basis. The wrapper inherits the `redeem()` and `withdraw()` functions from Openzeppelin's ERC4626, which internally call `_convertToShares()` and `_convertToAssets()` to calculate the amount of assets to return to users based on current total assets and supply.

As a result, a malicious user can break this 1:1 property by donating some assets (LBTCv shares) to the wrapper.

```
File: ERC4626Upgradeable.sol
252:     function _convertToShares(uint256 assets, Math.Rounding rounding) internal
 ↪  view virtual returns (uint256) {
253:         return assets.mulDiv(totalSupply() + 10 ** _decimalsOffset(),
 ↪  totalAssets() + 1, rounding);
254:     }
..SNIP..
259:     function _convertToAssets(uint256 shares, Math.Rounding rounding) internal
 ↪  view virtual returns (uint256) {
260:         return shares.mulDiv(totalAssets() + 1, totalSupply() + 10 **
 ↪  _decimalsOffset(), rounding);
261:     }
```

## Impact

The wrapper's 1:1 property/invariant will be broken, which might lead to unexpected results for users or off-chain components that utilized the wrapper.

## Code Snippet

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/stakeAndBake/depositor/veda/ERC4626VaultWrapper.sol#L27

## Tool Used

Manual Review

## Recommendation

Consider overwriting the `_convertToShares` and `_convertToAssets` functions so that the conversion between shares and assets is 1:1.

## Discussion

**555-andrew**

Agree. Fixed here: https://github.com/lombard-finance/smart-contracts/pull/352/commits/36c793fee3386b50ce04168d9bc3ed596c1df7e0

# Issue M-2: Redemption will be DOSed due to `shareUnlockPeriod` [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/4

This issue has been acknowledged by the team but won't be fixed at this time.

## Vulnerability Detail

The `shareUnlockPeriod` of the LBTCv vault is currently set to 24 hours (86400 seconds). When users redeem `ERC4626VaultWrapper` shares, the LBTCv vault shares on this wrapper will be transferred to the user.

However, if the share lock is still active, the wrapper cannot transfer its LBTCv vault shares due to the `beforeTransfer()` hook, as shown below, and a revert will occur.

https://etherscan.io/address/0x2ea43384f1a98765257bc6cb26c7131debdeb9b3/advanced#code#F1#L291

```
File: TellerWithMultiAssetSupport.sol
291:     function beforeTransfer(address from, address to, address operator) public
 ↪  view virtual {
292:         if (fromDenyList[from] || toDenyList[to] ||
 ↪  operatorDenyList[operator]) {
293:             revert TellerWithMultiAssetSupport__TransferDenied(from, to,
 ↪  operator);
294:         }
295:         if (shareUnlockTime[from] >= block.timestamp) revert
 ↪  TellerWithMultiAssetSupport__SharesAreLocked();
296:     }
```

The issue is that although the lock will be disabled/cooldowned after 24 hours, malicious users can keep depositing 1 wei, and reset/extend the `shareUnlockPeriod` of the wrapper, preventing anyone from redeeming.

When a user deposits, it executes the internal `_depositCustom()` function, which, in turn, invokes the `teller.deposit()` function.

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/stakeAndBake/depositor/veda/ERC4626VaultWrapper.sol#L243

```
File: ERC4626VaultWrapper.sol
228:     function _depositCustom(
229:         IERC20 token,
230:         address caller,
231:         address receiver,
```

```
232:         uint256 depositAmount,
233:         uint256 minimumMint
234:     ) internal returns (uint256) {
235:         ERC4626VaultWrapperStorage storage $ =
 ↪  _getERC4626VaultWrapperStorage();
236:
237:         SafeERC20.safeTransferFrom(token, caller, address(this),
 ↪  depositAmount);
238:
239:         // Give the vault the needed allowance.
240:         token.safeIncreaseAllowance($.teller.vault(), depositAmount);
241:
242:         // Deposit and obtain vault shares.
243:         uint256 shares = $.teller.deposit(token, depositAmount, minimumMint);
..SNIP..
250:     }
```

After the assets are deposited into the vault, the `_afterPublicDeposit()` hook in Line 370 below will be executed, and the `shareUnlockTime[wrapper]` will be extended by 24 hours.

https://etherscan.io/address/0x2ea43384f1a98765257bc6cb26c7131debdeb9b3/advanced#code#F1#L346

```
File: TellerWithMultiAssetSupport.sol
346:     function deposit(ERC20 depositAsset, uint256 depositAmount, uint256
 ↪  minimumMint)
..SNIP..
365:         } else {
366:             if (msg.value > 0) revert
 ↪  TellerWithMultiAssetSupport__DualDeposit();
367:             shares = _erc20Deposit(depositAsset, depositAmount, minimumMint,
 ↪  msg.sender);
368:         }
369:
370:         _afterPublicDeposit(msg.sender, depositAsset, depositAmount, shares,
 ↪  shareLockPeriod);
371:     }
```

https://etherscan.io/address/0x2ea43384f1a98765257bc6cb26c7131debdeb9b3/advanced#code#F1#L453

```
File: TellerWithMultiAssetSupport.sol
453:     function _afterPublicDeposit(
454:         address user,
455:         ERC20 depositAsset,
456:         uint256 depositAmount,
457:         uint256 shares,
458:         uint256 currentShareLockPeriod
459:     ) internal {
```

```
460:         shareUnlockTime[user] = block.timestamp + currentShareLockPeriod;
..SNIP..
467:     }
```

Sidenote: Even if `shareUnlockPeriod` is 0, it still results in single-block DOS. This is because the hook uses `>= block.timestamp` instead of `> block.timestamp`. If a block contains one deposit TX followed by one redemption TX. The redemption TX will revert.

## Impact

DOS. Users are unable to redeem their wrapper shares.

## Code Snippet

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/stakeAndBake/depositor/veda/ERC4626VaultWrapper.sol#L243

## Tool Used

Manual Review

## Recommendation

Consider using `TellerWithMultiAssetSupport.bulkDeposit()` instead of `TellerWithMultiAssetSupport.deposit()`.

The `bulkDeposit()` function deposits assets directly into the vault without triggering the _afterPublicDeposit hook.

https://etherscan.io/address/0x2ea43384f1a98765257bc6cb26c7131debdeb9b3/advanced#code#F1#L405

```
File: TellerWithMultiAssetSupport.sol
405:     function bulkDeposit(ERC20 depositAsset, uint256 depositAmount, uint256
 ↪  minimumMint, address to)
406:         external
407:         requiresAuth
408:         nonReentrant
409:         returns (uint256 shares)
410:     {
411:         if (!isSupported[depositAsset]) revert
 ↪  TellerWithMultiAssetSupport__AssetNotSupported();
412:
413:         shares = _erc20Deposit(depositAsset, depositAmount, minimumMint, to);
```

```
414:         emit BulkDeposit(address(depositAsset), depositAmount);
415:     }
```

## Discussion

**555-andrew**

Acknowledged. Fixed here:
https://github.com/lombard-finance/smart-contracts/pull/361/files

**555-andrew**

After some investigation and discussion we decided to use `deposit` function, but utilize a different `Teller` that has 0 lock time:
https://etherscan.io/address/0x4e8f5128f473c6948127f9cbca474a6700f99bab allowbreak #readContract Here is the pull request:
https://github.com/lombard-finance/smart-contracts/pull/365

# Issue M-3: Wrapper's internal accounting can be affected by Teller's refund mechanism [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/5

## Vulnerability Detail

The Teller supports the refundDeposit() function that allows `DEPOSIT_REFUNDER_ROLE` to revert/refund a deposit made earlier as long as the share lock period has not passed.

The problem is that if a single refund is issued against `ERC4626VaultWrapper`'s deposits, the accounting in `ERC4626VaultWrapper` will become incorrect.

1. At T0, the total supply of `ERC4626VaultWrapper` is 100 shares, while the total asset of `ERC4626VaultWrapper` is 100 LBTCv shares.
2. At T1, Bob deposits some assets via the `ERC4626VaultWrapper`, and the Teller transfers 10 LBTCv shares to the wrapper. This, in turn, mint 10 wrapper shares to Bob.
3. The state of the wrapper becomes the total supply of `ERC4626VaultWrapper` is 110 shares, while the total asset of `ERC4626VaultWrapper` is 110 LBTCv shares
4. Someone with `DEPOSIT_REFUNDER_ROLE` executes a refund against the earlier deposit. 10 LBTCv shares will be burned from wrapper, and it will transfer the assets that Bob deposited back to the wrapper.
5. The state of the wrapper becomes the total supply of `ERC4626VaultWrapper` is 110 shares, the total asset of `ERC4626VaultWrapper` is 100 LBTCv shares, and some refunded assets (LBTC, WBTC).
6. At this point, the 1:1 share invariant is broken. As a side effect, some holders of wrapper shares might not be able to redeem since there is only 100 LBTCv share backinig 110 wrapper shares (shortfall of 10 LBTCv shares)

## Impact

The accounting in `ERC4626VaultWrapper` will become incorrect. 1:1 share invariant will be broken, and some holders of wrapper shares might not be able to redeem.

## Code Snippet

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/stakeAndBake/depositor/veda/ERC4626VaultWrapper.sol#L25

## Tool Used

Manual Review

## Recommendation

Ensure that the `Teller.refundDeposit()` function is never called against the wrapper address. Otherwise, the wrapper's internal accounting will be incorrect.

If a refund cannot be prevented, consider having a function (e.g., `recoverERC20()`) to retrieve/sweep the refunded assets (LBTC, WBTC) in the wrapper. After a refund, the 1:1 share invariant will be broken because the total number of LBTCv shares is less than the total supply/shares. In this case, the swept assets (LBTC, WBTC) might have to be manually swapped into LBTCv shares and then redeposited back into the wrapper to ensure 1:1 property is upheld.

## Discussion

**555-andrew**

`Teller` is being managed by Veda, we do not have control over it. Currently `refund` is disabled on the `Teller` so it should be okay. But to have some protection against this issue we have implemented `rescueERC20()` function:
https://github.com/lombard-finance/smart-contracts/pull/358/files

# Issue L-1: Unused `ReentrancyGuardUpgradeable` in the wrapper [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/6

## Vulnerability Detail

The `ReentrancyGuardUpgradeable` is not being utilized in this wrapper.

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/stakeAndBake/depositor/veda/ERC4626VaultWrapper.sol#L31

```
File: ERC4626VaultWrapper.sol
25: contract ERC4626VaultWrapper is
26:     IDepositor,
27:     ERC4626Upgradeable,
28:     ERC20PausableUpgradeable,
29:     ERC20PermitUpgradeable,
30:     AccessControlDefaultAdminRulesUpgradeable,
31:     ReentrancyGuardUpgradeable
32: {
```

## Impact

N/A

## Code Snippet

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/stakeAndBake/depositor/veda/ERC4626VaultWrapper.sol#L31

## Tool Used

Manual Review

## Recommendation

Consider either removing the `ReentrancyGuardUpgradeable` or adding the `nonReentrant` modifier to the public methods of the Openzeppelin ERC4626 (.e.g, `deposit`, `mint`, `withdraw`, `redeem`).

## Discussion

**555-andrew**

Fixed here: https://github.com/lombard-finance/smart-contracts/pull/361

# Issue L-2: Non-compliance to EIP4626 [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/8

This issue has been acknowledged by the team but won't be fixed at this time.

## Vulnerability Detail

Following is an extract from EIP4626 related to this for reference:

> maxDeposit MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily) it MUST return 0.

The Openzeppelin's ERC4626Upgradeable has the following behavior:

- `maxDeposit` returns `type(uint256).max`
- `maxMint` returns `type(uint256).max`
- `maxWithdraw` returns `convertToAssets(balanceOf(owner))`
- `maxRedeem` returns `balanceOf(owner)`

### Instance 1

The wrapper is pausable. When the wrapper is paused, no minting or redeeming of wrapper shares can be executed. If the wrapper must strictly comply with EIP4626, the functions mentioned above should return 0 when the wrapper is paused to reflect that.

```
File: ERC4626VaultWrapper.sol
25: contract ERC4626VaultWrapper is
26:     IDepositor,
27:     ERC4626Upgradeable,
28:     ERC20PausableUpgradeable,
29:     ERC20PermitUpgradeable,
30:     AccessControlDefaultAdminRulesUpgradeable,
31:     ReentrancyGuardUpgradeable
32: {
```

### Instance 2

The denylist in the `beforeTransfer` hook in `TellerWithMultiAssetSupport` also affects this behavior. If either the wrapper or the user is blacklisted, the minting/depositing/redemption/withdrawals are considered disabled and should return 0.

```
File: TellerWithMultiAssetSupport.sol
291:     function beforeTransfer(address from, address to, address operator) public
 ↪  view virtual {
```

```
292:         if (fromDenyList[from] || toDenyList[to] ||
 ↪  operatorDenyList[operator]) {
293:             revert TellerWithMultiAssetSupport__TransferDenied(from, to,
 ↪  operator);
294:         }
295:         if (shareUnlockTime[from] >= block.timestamp) revert
 ↪  TellerWithMultiAssetSupport__SharesAreLocked();
296:     }
297:
```

### Instance 3

The `accountant` in `TellerWithMultiAssetSupport`, which is used in `_erc20Deposit`, can also be paused as the rate to calculate shares is done using `getRateInQuoteSafe`, which checks for a paused `accountant` and can revert. In this case, the deposit/mint is considered disabled and should return 0.

```
File: TellerWithMultiAssetSupport.sol
440:     function _erc20Deposit(ERC20 depositAsset, uint256 depositAmount, uint256
 ↪  minimumMint, address to)
441:         internal
442:         returns (uint256 shares)
443:     {
444:         if (depositAmount == 0) revert
 ↪  TellerWithMultiAssetSupport__ZeroAssets();
445:         shares = depositAmount.mulDivDown(ONE_SHARE,
 ↪  accountant.getRateInQuoteSafe(depositAsset));
446:         if (shares < minimumMint) revert
 ↪  TellerWithMultiAssetSupport__MinimumMintNotMet();
447:         vault.enter(msg.sender, depositAsset, depositAmount, to, shares); //
 ↪  @audit-info "to" => wrapper
448:     }
```

```
File: AccountantWithRateProviders.sol
388:     function getRateInQuoteSafe(ERC20 quote) external view returns (uint256
 ↪  rateInQuote) {
389:         if (accountantState.isPaused) revert
 ↪  AccountantWithRateProviders__Paused();
390:         rateInQuote = getRateInQuote(quote);
391:     }
```

## Impact

Non-compliance with EIP4626.

## Code Snippet

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/stakeAndBake/depositor/veda/ERC4626VaultWrapper.sol#L28

## Tool Used

Manual Review

## Recommendation

Review if strict compliance with EIP4626 is required for the wrapper contract. If so, update the implementation of `max*` function to return the correct value when the feature is disabled.

## Discussion

**555-andrew**

We acknowledge this deviation from the ERC4626 and consider it acceptable.

# Issue L-3: References in Wrapper are declared but never used, indicating a missing zero asset check [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/13

## Summary

Some code references, namely imports and custom errors are declared, but never used. In the case of `ZeroAssets()`, depending on the team's intention, this may indicate a missing check in the deposit flow.

## Vulnerability Detail

The references in question are:

- Imports: `IERC4626`, `Math`
- Errors: `ZeroAssets()`, `NotImplemented()`

Declared inside the `ERC4626VaultWrapper`. In the case of `ZeroAssets`, it should be used to guard against 0 value deposits in the vault which is a common check and currently not implemented in the public deposit functions.

## Impact

Potential missing checks and unnecessary declarations

## Code Snippet

```
/// @dev error thrown when the passed depositAmount is zero
    error ZeroAssets();
```

## Tool Used

Manual Review

## Recommendation

Clean up unused references and add a zero asset check in public deposit functions

## Discussion

**555-andrew**

`IERC4626` import and `ZeroAssets()`, `NotImplemented()` removed here: https://github.com/lombard-finance/smart-contracts/pull/361/files `Math` import is required for custom versions of `_convertToAssets()` and `_convertToShares()`

# Issue L-4: Pausing functionality in the Wrapper is unclear and forces users to waste gas on failed deposits [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/15

## Summary

The `ERC4626VaultWrapper` implements a pausing functionality on all vault token transfers by inheriting from `ERC20PausableUpgradeable` and exposing the privileged `pause` and `unpause` functions. The pause however is only enforced in the internal ERC20 `_update` function which exists up the inheritance chain and reverts only at the last step of the deposit process creating ambiguity about the code and forcing users who attempt to deposit when the vault is paused to waste more gas than necessary.

## Vulnerability Detail

Pausing in the `ERC4626VaultWrapper` pertains only to the internal `ERC20PausableUpgradeable._update` function which limits the transfers of the share token when the contract is paused, thus limiting deposits and redemptions.

```
/**
 * @dev See {ERC20-_update}.
 *
 * Requirements:
 *
 * - the contract must not be paused.
 */
function _update(address from, address to, uint256 value) internal virtual override
 ↪  whenNotPaused {
    super._update(from, to, value);
}
```

Given this is a contract up the inheritance chain and no explicit pausing modifiers are used in any of the setter or public functions, it would be good to document it, along with which functionality is expected to not work upon a pause, to reduce ambiguity.

In addition, on the deposit flows specifically, since update is called later in the execution trace when the actual wrapper shares get minted to the caller, it means that users will waste some more gas before reverting when the contract is paused compared to if the modifiers were used at the start of the function.

```
    function _depositCustom(
        IERC20 token,
```

```
        address caller,
        address receiver,
        uint256 depositAmount,
        uint256 minimumMint
    ) internal returns (uint256) {
        ERC4626VaultWrapperStorage storage $ = _getERC4626VaultWrapperStorage();

        SafeERC20.safeTransferFrom(token, caller, address(this), depositAmount);

        // Give the vault the needed allowance.
        token.safeIncreaseAllowance($.teller.vault(), depositAmount);
        uint256 shares = $.teller.deposit(token, depositAmount, minimumMint);

        // we mint shares 1:1 with minted veda vault shares
        // revert due to pause happens here
@>      _mint(receiver, shares);

        emit Deposit(caller, receiver, depositAmount, shares);

        return shares;
    }
```

## Impact

Reduced clarity on pausing and wasted gas on failed deposits

## Code Snippet

-

## Tool Used

Manual Review

## Recommendation

Update the natspec to reflect the pausing behavior and which functions are expected to be paused. Consider explicitly adding the `whenNotPaused` modifier on the public deposit functions.

## Discussion

**555-andrew**

Fixed here: https://github.com/lombard-finance/smart-contracts/pull/361/files

# Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.